Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Akış diyagramında üs alma ve fonksiyonlar
  - Üstel gösterim (ör. `2^5`) akış diyagramında kullanılmaz. Bunun yerine for döngüsünde çarpma tekrarlanır: 2^5 için 2 sayısı 5 kez çarpılır.
  - Dört işlem, for, while ve if aslında tüm problemleri çözmeye yeter. Fonksiyonlar bunların özelleştirilmiş hâlleridir.
  - Akış diyagramında karşılığı olmayan işlemler için istisnai olarak fonksiyon kullanılır. Örneğin karekök için `sqrt` fonksiyonu çağrılır.
- Algoritma 9: 365 günlük sıcaklıklarda en sıcak ve en soğuk gün (ekrandan, 00:28:00)
  - START → mesaj → for `I ← 1` … `365`: giriş `T[I]` (sıcaklıklar T dizisine okunur).
  - Başlangıç değerleri dizinin ilk elemanından alınır: `enbuyuk = T[1]`, `enkucuk = T[1]`, `enSicakGun = 1`, `enSogukGun = 1`.
  - for `I ← 2` … `365` (ilk eleman zaten başlangıç olduğu için 2'den başlanır):
    - `T[I] >= enbuyuk` doğruysa: `enbuyuk = T[I]`, `enSicakGun = I`.
    - Yanlışsa: `T[I] <= enkucuk` kontrol edilir. Doğruysa `enkucuk = T[I]`, `enSogukGun = I`.
  - Döngüden sonra: `'En sıcak gün=', enSicakGun, enbuyuk, 'En soğuk gün=', enSogukGun, enkucuk` yazdırılır → STOP.
  - Gün indisini tutan değişkene yeni günün indisi atanır (`enSicakGun = I`). Sayaç gibi artırılmaz.
  - En büyük ve en küçük aynı döngüde bulunmalıdır. İki ayrı döngü kullanmak yanlış değildir, ama diziyi iki kez (2 × 365) gezmek gereksiz iş demektir.
  - En küçük kontrolü, en büyük kontrolünün false kolunda yapılır. Bir değer en büyükten büyükse en küçükten küçük olamaz. İki if'i art arda koymak gereksiz kontrol yapar.
  - Eşitlik (>=, <=) kullanıldığı için aynı sıcaklığa sahip birden fazla gün varsa en son gelen gün sonuç olur. Tüm günler istenseydi bu günlerin bir dizide saklanması gerekirdi.
  - Dizi sıralı verilseydi tüm diziyi gezmeye gerek kalmazdı: ilk ve son eleman en küçük ve en büyük olurdu.
- Değişken isimleri
  - `count1`, `count2`, `max` gibi belirsiz isimler yerine sakladığı bilgiyi anlatan isimler kullanılmalıdır: `enSicakGun`, `enSogukGun`, `enbuyuk`, `enkucuk`.
- Algoritma 10: Dizide tekrar eden eleman olup olmadığını bulma
  - Problem: N elemanlı bir dizide tekrar eden bir eleman varsa "Not unique", yoksa "Unique" yazdırılacaktır. Örnek: `1, 3, 3, 5` tekrarlıdır. `6, 2, 1, 3` tekrarsızdır (unique).
  - Kağıt üzerinde fikir: ilk eleman kendisinden sonraki tüm elemanlarla karşılaştırılır, sonra ikinci eleman kendisinden sonrakilerle, … sondan bir önceki elemana kadar.
  - İç içe for ile çözüm: dış döngü `I = 1 ... N-1`, iç döngü `J = I+1 ... N`, `Dataset[I] = Dataset[J]` ise `unique = 0`. Bu çözüm tekrar bulunduktan sonra da gereksiz yere sonuna kadar döner.
  - Doğru çözüm: adım sayısı belli olmadığı için (ilk karşılaştırmada da bitebilir, sonuna kadar da gidebilir) iç içe while kullanılır (ekrandan, 01:21:30):

```
unique = 1, I = 1
while (I < N AND unique = 1)
    J = I + 1
```

```
        while (J <= N AND unique = 1)
            Dataset[I] = Dataset[J] ise unique = 0
            J = J + 1
        I = I + 1
    unique = 1 ise 'Unique', değilse 'Not unique' → STOP
```

  - While döngüsünde indisler elle artırılmalıdır (`J = J + 1`, `I = I + 1`). For döngüsünde ise indis otomatik artar.
  - Analiz (`1, 3, 5, 7`): tüm (I, J) çiftleri karşılaştırılır, hiçbiri eşit olmaz ve sonuç "Unique" çıkar. `1, 1, 3, 5` için ilk karşılaştırmada eşitlik bulunur, `unique = 0` olur ve iki döngüden de hemen çıkılır.
  - Bu "kaba kuvvet" (brute force) çözümdür. İç içe iki döngü nedeniyle karmaşıklık N^2'ye çıkar. Daha verimli çözümler ilerideki derslerde görülecektir.
- Mantıksal VE / VEYA (logical AND / OR)
  - Bir döngünün devam etmesi için iki koşulun birlikte sağlanması gerekiyorsa koşullar `AND` (mantıksal ve) ile bağlanır: `I < N AND unique = 1`.
  - Koşullardan birinin sağlanması yeterliyse `OR` (mantıksal veya) kullanılır.
  - C'de bu operatörler `&&` (ve) ve `||` (veya) olarak yazılır. Ağırlıklı olarak while koşullarında, gerektiğinde if içinde de kullanılır.
- Karakter dizileri (ekrandan, 01:34:20)
  - Tam sayı dizisinde (array of integers) `1, 2, 3, 4` bellekte 1, 2, 3, 4 değerleriyle saklanır.
  - Karakter dizisinde (array of characters) karakterlerin ASCII tablosundaki sayısal karşılıkları saklanır. `'1', '2', '3', '4'` bellekte 49, 50, 51, 52 olarak durur.
  - `'0'...'9'`, `'a'...'z'` ve `'A'...'Z'` karakterleri tabloda art arda gelir. Bu bir kuraldır, ama başlangıç değerleri ezberlenmez.
  - Karakter sabitleri arasındaki fark kullanılır: `'1' - '0' = 1`, `'9' - '0' = 9`.
- Alıştırma (Algoritma 11): N rakamdan oluşan karakter dizisini tam sayıya çevirme
  - `['1', '2', '3', '4']` → 1234, `['3', '0', '5']` → 305, `['1']` → 1.
  - Yalnızca karakter sabitleri kullanılacaktır. ASCII değerleri doğrudan kullanılmayacaktır.
  - Dizinin 0 ile başlamadığı varsayılır. Çözüm bir sonraki derste yapılacaktır.

Hocanın Özellikle Vurguladığı Kısımlar

- İstenmeyen fazlalıkları eklemeyin
  - Bir çözümde istenen problemle ilgisiz ek işler yapmak, karşı tarafta talebin anlaşılıp anlaşılmadığı konusunda tereddüt yaratır. Problemle ilgili anlamlı ek durumlar (örn. negatif değerlerin de ele alınması) ise olumlu karşılanır.
- For döngüsünden zorla çıkılmaz
  - `break` kullanmak ya da döngü indisini döngü içinde değiştirerek (ör. `I = N`) döngüyü bitirmek kodun akışını bozar ve istenmez. Adım sayısı belli değilse while kullanılmalı ve çıkış koşulu while koşuluna yazılmalıdır.
  - For döngüsünün içindeki bir if'in sonuna STOP konulmaz.
- Gereksiz döngü ve kontrolden kaçının
  - İşler mümkünse tek döngüde bitirilmelidir. Mantıksal olarak imkânsız bir durum için ikinci kez kontrol yapılmamalıdır.
- ASCII değerlerini kullanmayın
  - Hesaplamalarda karakterlerin sayısal değerleri yerine karakter sabitleri ve aralarındaki farklar kullanılmalıdır.
- Akış diyagramında programlama dili kısayolları kullanılmaz
  - Değişkene metin atayıp döngüden çıkmak gibi dile özgü kısayollar yerine sayısal bayrak (`unique = 1/0`) ve if kontrolü kullanılır.

Kısa Tekrar Notları

- Üs → for ile çarpma tekrarı. Karekök → `sqrt` (istisna).
- En büyük/en küçük: başlangıç = ilk eleman, döngü 2'den başlar. En küçük kontrolü en büyük kontrolünün false kolunda yapılır.
- Gün indisi için `enSicakGun = I` (atama, sayaç değil).

- Tekrar kontrolü: iç içe while, `I < N AND unique = 1`, `J <= N AND unique = 1`. Eşitlik bulunca `unique = 0`.
- While'da indis elle artırılır. For'da otomatik artar.
- `break` ve indisi elle değiştirmek yok.
- AND = ikisi de sağlanmalı. OR = biri yeterli.
- `'1'` bellekte 49'dur. `'1' - '0' = 1`.

Detaylı Açıklamalar

Ders, önceki derste alıştırma olarak verilen en sıcak ve en soğuk gün probleminin çözümüyle başlar. 365 günün sıcaklıkları bir diziye okunur. Ardından en büyük ve en küçük sıcaklık ile bunların hangi günde olduğu için dört değişkene dizinin ilk elemanı ve 1 değeri verilir. İlk eleman başlangıç olarak kullanıldığı için karşılaştırma döngüsü ikinci günden başlar. Her günün sıcaklığı önce en büyükle karşılaştırılır. Büyük ya da eşitse en büyük değer ve günü güncellenir. Değilse aynı değer en küçükle karşılaştırılır ve gerekirse en küçük değer ve günü güncellenir. Hoca, en küçük kontrolünün yalnızca en büyük kontrolü başarısız olduğunda yapılmasının neden mantıklı olduğunu açıklar: bir değer en büyükten büyükse en küçükten küçük olması mümkün değildir. Ayrıca en büyük ve en küçük değerin iki ayrı döngüde bulunmasının doğru sonuç vereceğini, ancak diziyi iki kez gezmenin gereksiz olduğunu belirtir.

Değişken isimleri konusunda hoca öğrencilerden `count1` gibi belirsiz isimler yerine `enSicakGun` gibi anlamlı isimler kullanmalarını ister. Kısa algoritmalarda belirsiz isimler sorun çıkarmayabilir, ancak algoritma uzadıkça okunabilirlik önemli hâle gelir. Eşitlik operatörlerinin kullanılması nedeniyle aynı sıcaklıkta birden fazla gün varsa sonuçta en son gelen günün görüneceği, bunun yanlış olmadığı, ancak tüm günler istenseydi bir diziye ihtiyaç duyulacağı belirtilir.

İkinci problem bir dizide tekrar eden eleman olup olmadığını bulmaktır. Önce kağıt üzerindeki yaklaşım konuşulur: her eleman kendisinden sonraki tüm elemanlarla karşılaştırılır. Öğrencilerin çoğu bunu iç içe for döngüleriyle çözmüştür. Hoca bu çözümün doğru sonuç verdiğini, ancak tekrar bulunduktan sonra da döngülerin sonuna kadar dönmeye devam ettiğini gösterir. Bu döngülerden `break` ile ya da indisi değiştirerek çıkmak ise kodun akışını bozan, istenmeyen bir kullanımdır. Problemin adım sayısı girdiye göre değiştiği için doğru araç while döngüsüdür.

While ile çözümde bir `unique` bayrağı 1 olarak başlatılır. Dış döngü I, N'den küçük olduğu ve `unique` 1 olduğu sürece döner. İç döngü J = I + 1'den başlar ve J, N'den küçük ya da eşit olduğu ve `unique` 1 olduğu sürece döner. İki eleman eşitse bayrak 0 yapılır ve bu, her iki döngünün koşulunu da bozarak döngülerden doğal biçimde çıkılmasını sağlar. Hoca, while döngülerinde indislerin elle artırılması gerektiğini özellikle vurgular. Burada ilk kez iki koşulun mantıksal VE ile bağlandığı bir döngü koşulu kullanılır. Analizde tekrarsız `1, 3, 5, 7` dizisi için tüm karşılaştırmalar adım adım izlenir ve sonucun "Unique" olduğu gösterilir. `1, 1, 3, 5` dizisi için ise ilk karşılaştırmada eşitlik bulunup döngülerden hemen çıkıldığı görülür. Hoca bu çözümün kaba kuvvet yaklaşımı olduğunu ve iç içe döngüler nedeniyle karmaşıklığın N^2 olduğunu, daha verimli yöntemlerin ileride öğrenileceğini belirtir.

Ders sonunda karakter dizilerine giriş yapılır. Tam sayı dizisinde sayılar doğrudan saklanırken karakter dizisinde karakterlerin ASCII karşılıkları saklanır. Örneğin `'1'` karakteri bellekte 49 değeriyle tutulur. Rakam ve harf karakterleri tabloda art arda geldiği için karakter sabitleri arasındaki farklar hesaplamada kullanılabilir: `'1' - '0'` sayısal olarak 1'dir. Bu bilgiyle çözülmek üzere, rakam karakterlerinden oluşan bir diziyi tek bir tam sayıya (örneğin `'3','0','5'` → 305) çeviren algoritmanın akış diyagramı alıştırma olarak verilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.